

Day 12: Arrays, Buffers & Static Allocation

Platform: nRF52840 | RTOS: Zephyr RTOS

Learning Objectives

Fixed-Size Arrays vs Dynamic Allocation

In embedded, **dynamic allocation** (`new` , `malloc`) is dangerous:

- **Non-deterministic timing** — `malloc` can take variable time
- **Fragmentation** — repeated alloc/free fragments the heap, eventually causing allocation failure
- **Hard to debug** — heap corruption causes subtle, intermittent bugs

Instead, prefer **static allocation**: arrays sized at compile time, allocated in `.bss` or `.data` :

```
static uint8_t uart_rx_buf[256]; // Always 256 bytes, always available
static uint8_t spi_tx_buf[64];   // Known size, deterministic
```

std::array in Embedded

`std::array` is a zero-overhead wrapper around a C array that adds:

- `.size()` — no magic numbers
- `.at()` — bounds-checked access (in debug mode)
- Works with range-for loops and STL algorithms

```
#include <array>
std::array<uint8_t, 16> tx_buf{}; // Zero-initialized
tx_buf.fill(0xFF);               // Set all elements
tx_buf[0] = 0xAA;
```

In release builds (`-O2`), `std::array` compiles to identical code as a raw C array.

Static Memory Pools (k_mem_slab)

Zephyr's `k_mem_slab` provides **deterministic fixed-size block allocation** — like `malloc` but with a bounded pool, fixed block size, and O(1) alloc/free:

```
K_MEM_SLAB_DEFINE(msg_slab, sizeof(Message), 8, 4); // 8 blocks of sizeof(Message)
void *block;
k_mem_slab_alloc(&msg_slab, &block, K_NO_WAIT); // Get a block
k_mem_slab_free(&msg_slab, block);              // Return it
```

Use this when you need dynamic-looking allocation but can't afford heap fragmentation.

Ring Buffer Basics

A **ring buffer** (circular buffer) is a fixed-size FIFO where the write pointer wraps around to the start when it reaches the end. Essential for:

- **UART RX buffer** — ISR writes, main thread reads
- **ADC sample buffer** — DMA writes, processing thread reads
- **Logging buffer** — log writes, flush reads

K_MEM_SLAB for Embedded Allocation

Use `k_mem_slab` instead of `new` / `delete` for message objects, packet buffers, and any other object that needs to be created and destroyed at runtime. The slab guarantees:

- Allocation always succeeds (or fails deterministically with `K_NO_WAIT`)
- No fragmentation
- O(1) performance

Avoiding Heap in Embedded

Zephyr disables the system heap by default (`CONFIG_HEAP_MEM_POOL_SIZE=0`). If you must use `new` / `delete` , define a bounded heap with `CONFIG_HEAP_MEM_POOL_SIZE=N` and use `K_HEAP_DEFINE` for application heaps. Never use unlimited heap in production firmware.

Practice Tasks

1. Implement `std::array` as an ADC sample buffer — fill and drain it
 2. Use `K_MEM_SLAB_DEFINE` to create a pool of 4 message objects — allocate and free them in a loop
 3. Implement a simple ring buffer (head/tail indices) for `uint8_t` — test with producer/consumer
 4. Set `CONFIG_HEAP_MEM_POOL_SIZE=0` and try to `new` an object — observe the linker error
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day12
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day12/main.cpp](#)